Practical- 03

Title of the Assignment: Tower of Hanoi using State Space Search

Problem statement: Implement a state-space search technique to solve the Tower of Hanoi problem and display the sequence of legal moves from the initial state to the goal state.

Objective:

To understand problem formulation using states, operators, initial state, goal state and path cost.

To represent the Tower of Hanoi as a state-space search problem.

To implement a search procedure that generates only legal successor states and finds a solution path.

Theory:

State space search:

State-space search is a problem-solving method in which every possible configuration of a problem is represented as a state. Operators are applied to a state to generate successor states. The search begins at an initial state and continues until a state satisfying the goal condition is reached. A solution is a sequence of operators that transforms the initial state into the goal state.

For the Tower of Hanoi, the state is represented by the arrangement of disks on the three pegs. A legal move transfers the top disk from one peg to another peg only when the destination peg is empty or its top disk is larger than the moved disk. The initial state has all disks arranged on the source peg in decreasing order of size, and the goal state has all disks arranged on the destination peg in the same order.

Each legal move has unit cost. Therefore, the cost of a solution is the number of moves. For n disks, the optimal solution requires $2^n - 1$ moves. A state can be stored as a tuple of three stacks, for example, ((3, 2, 1), (), ()) for three disks, where the last element of each stack is its top disk.

Problem formulation:

Initial state: All disks are placed on peg A, with the largest disk at the bottom and the smallest disk at the top.

Goal state: All disks are placed on peg C in the same order, with pegs A and B empty.

Operators: Move the top disk from peg A, B or C to another peg whenever the move obeys the size constraint.

Path cost: One unit for every legal disk movement.

Search strategy: Breadth-first search may be used to guarantee the shortest sequence for a finite state space. Depth-first search with cycle checking may also be used to produce a valid solution.

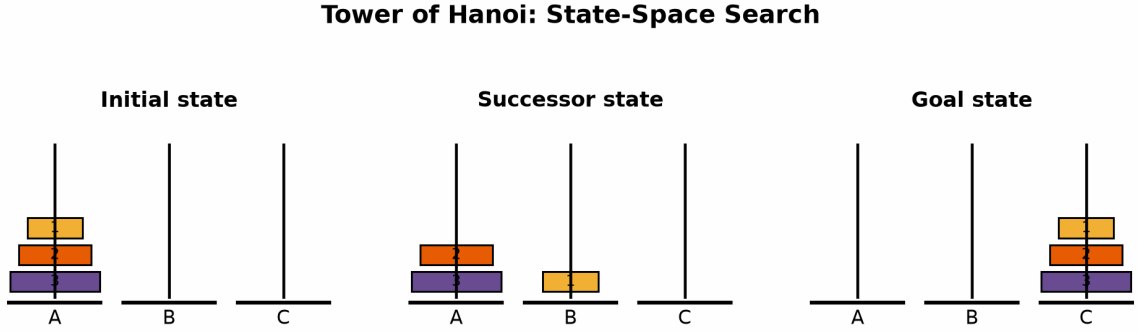


Figure 1: Tower of Hanoi states and legal state transitions

Algorithm:

Step 1: Start.

Step 2: Read the number of disks n and define the source peg, auxiliary peg and destination peg.

Step 3: Create the initial state with all n disks on the source peg.

Step 4: Define the goal test as: all disks are on the destination peg in decreasing order from bottom to top.

Step 5: Insert the initial state into a queue and mark it as visited.

Step 6: Remove the next state from the queue.

Step 7: If the selected state satisfies the goal test, reconstruct and display the path using parent pointers, then stop.

Step 8: Generate every ordered pair of source and destination pegs.

Step 9: For each pair, check whether the source is non-empty and the move is legal.

Step 10: Generate the successor state by moving the top disk and assign a path cost one greater than the current state.

Step 11: If the successor has not been visited, store its parent and move information, mark it visited and insert it into the queue.

Step 12: Repeat Steps 6–11 until the goal state is reached.

Step 13: Display the number of states explored and the complete sequence of disk movements.

Step 14: Stop.

Illustration for three disks:

Initial state: A = [3, 2, 1], B = [ ], C = [ ]

Goal state: A = [ ], B = [ ], C = [3, 2, 1]

The expected optimal move count is $2^3 - 1 = 7$ moves. The program should print each move, such as “Move disk 1 from A to C”, and should never place a larger disk on a smaller disk.

Expected output:

The output should contain the initial state, each selected legal move, the resulting state after every move, the final goal state and the total number of moves. For n = 3, the total number of moves in an optimal solution should be 7.

Conclusion:

The Tower of Hanoi was formulated and solved as a state-space search problem. The implementation demonstrates how states, legal operators, goal testing, visited-state checking and parent tracking can be combined to obtain a valid or shortest solution path.

Oral questions:

1. What is meant by a state space in artificial intelligence?
2. What are the initial state and goal state in the Tower of Hanoi problem?
3. Which moves are legal in the Tower of Hanoi?
4. Why is cycle checking useful in state-space search?
5. How is the path cost calculated in this problem?
6. What is the minimum number of moves required for n disks?
7. Why does breadth-first search find an optimal solution when every move has equal cost?
8. What is the difference between a state and an operator?
9. How can parent pointers be used to reconstruct the solution path?
10. What is the time and space complexity of the recursive Tower of Hanoi solution?

References:

[1] Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, section on problem-solving agents and search.

[2] Thomas H. Cormen et al., Introduction to Algorithms, section on recursive solutions and combinatorial state spaces.